

Lecture 011

Neural networks

Edward Rubin

Admin

Today

Topics: We will start an introduction to neural networks:

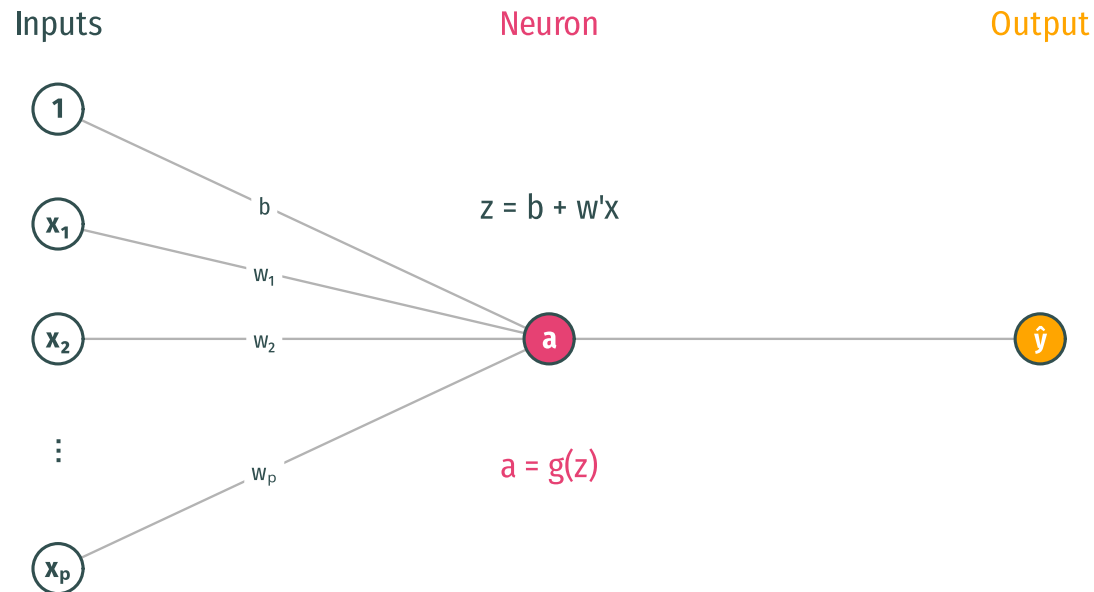
- anatomy of a single neuron,
- logistic regression as a one-neuron network,
- activation functions and hidden layers,
- a brief overview of training,
- the return of MNIST.

Reading and resources

- *Mostly:* *Neural Networks and Deep Learning*, Ch. 1 (some 2 + 3),
- *Also helpful:* ISL Ch. 11,
- *Fun, interactive tool:* TensorFlow's [Neural Network Playground](#).

Neural networks

(Neural) Intuition



Anatomy **Neurons** are the building blocks of neural networks.

- take **in** a weighted sum of inputs, $w'x = w_1x_1 + w_2x_2 + \dots + w_px_p$,
- apply a **bias**, b , that shifts the sum, $z = b + w'x$,
- pass z through an **activation function** to produce **output**, $a = g(z)$.

Neural networks

History It all started with the perceptron

The **perceptron** use a **hard-threshold** activation:

$$a = \begin{cases} 1 & \text{if } w'x \geq T \iff w'x + b \geq 0 \\ 0 & \text{if } w'x < T \iff w'x + b < 0 \end{cases}$$

The intuition literally came from neurons firing.

When the *weighted sum of inputs* ($w'x$) exceeds a **threshold** (T), **fire!**

Intuition: Compare weighted *evidence* against the *burden of proof*.

Neural networks

History It all started with the perceptron

The **perceptron** use a **hard-threshold** activation:

$$a = \begin{cases} 1 & \text{if } w'x \geq T \iff w'x + b \geq 0 \\ 0 & \text{if } w'x < T \iff w'x + b < 0 \end{cases}$$

The intuition literally came from neurons firing.

When the *weighted sum of inputs* ($w'x$) exceeds a **threshold** (T), **fire!**

Intuition: Compare weighted *evidence* against the *burden of proof*.

We translate **threshold** into **bias** ($b = -T$) to get a more "standard" form.

Neural networks

What do the weights and bias do?

$$z = b + \sum_{j=1}^p w_j x_j$$

The weighted sum does two jobs:

- the **weights** say which inputs matter and in which direction,
- the **bias** shifts the cutoff left or right.

Neural networks

What do the weights and bias do?

$$z = b + \sum_{j=1}^p w_j x_j$$

The weighted sum does two jobs:

- the **weights** say which inputs matter and in which direction,
- the **bias** shifts the cutoff left or right.

Even with its simple structure, the neuron is capable of learning

- to set the weights and bias to find a good decision boundary,
- to 1 when $z \geq 0$ and 0 when $z < 0$,
- to separate our two classes.

Neural networks

Too binary?

Binary activation may be *intuitive*, but it's *not great for learning*.

Problem We need to *learn* the weights and bias (w, b) .

- When **training**, we want to *adjust* w and b to reduce errors.
- If we make *small changes* to w or b , binary activation may change *a lot*.

⇒ training with binary activations can be *unstable* and *inefficient*.

Neural networks

Too binary?

Solution Keep what we like about the perceptron

- the **weighted-sum of inputs shifted by bias**,
- **restricting the output** to a certain range ($\in [0, 1]$ for binary classification).

but replace the hard step with a **smooth, differentiable activation**.

Q Any ideas? You already know one!

Neural networks

Too binary?

Solution Keep what we like about the perceptron

- the **weighted-sum of inputs shifted by bias**,
- **restricting the output** to a certain range ($\in [0, 1]$ for binary classification).

but replace the hard step with a **smooth, differentiable activation**.

Q Any ideas? *You already know one!*

The **sigmoid neuron** keeps the same weighted-sum idea and replaces the hard step with a differentiable activation—the **sigmoid function**.

Neural networks

Logistic regression or a tiny neural network?

Logistic regression For a binary outcome Y

$$\Pr(Y = 1 \mid X = x) = \sigma(\beta_0 + x'\beta) = \sigma(w'x + b)$$

where $\sigma(z) = \frac{1}{1 + e^{-z}}$.

Also **Sigmoid neuron** with

- a **weighted sum of inputs** and a **bias**,
- a **sigmoid** activation function,
- one output.

Neural networks

Logistic regression or a tiny neural network?

Logistic regression For a binary outcome Y

$$\Pr(Y = 1 \mid X = x) = \sigma(\beta_0 + x'\beta) = \sigma(w'x + b)$$

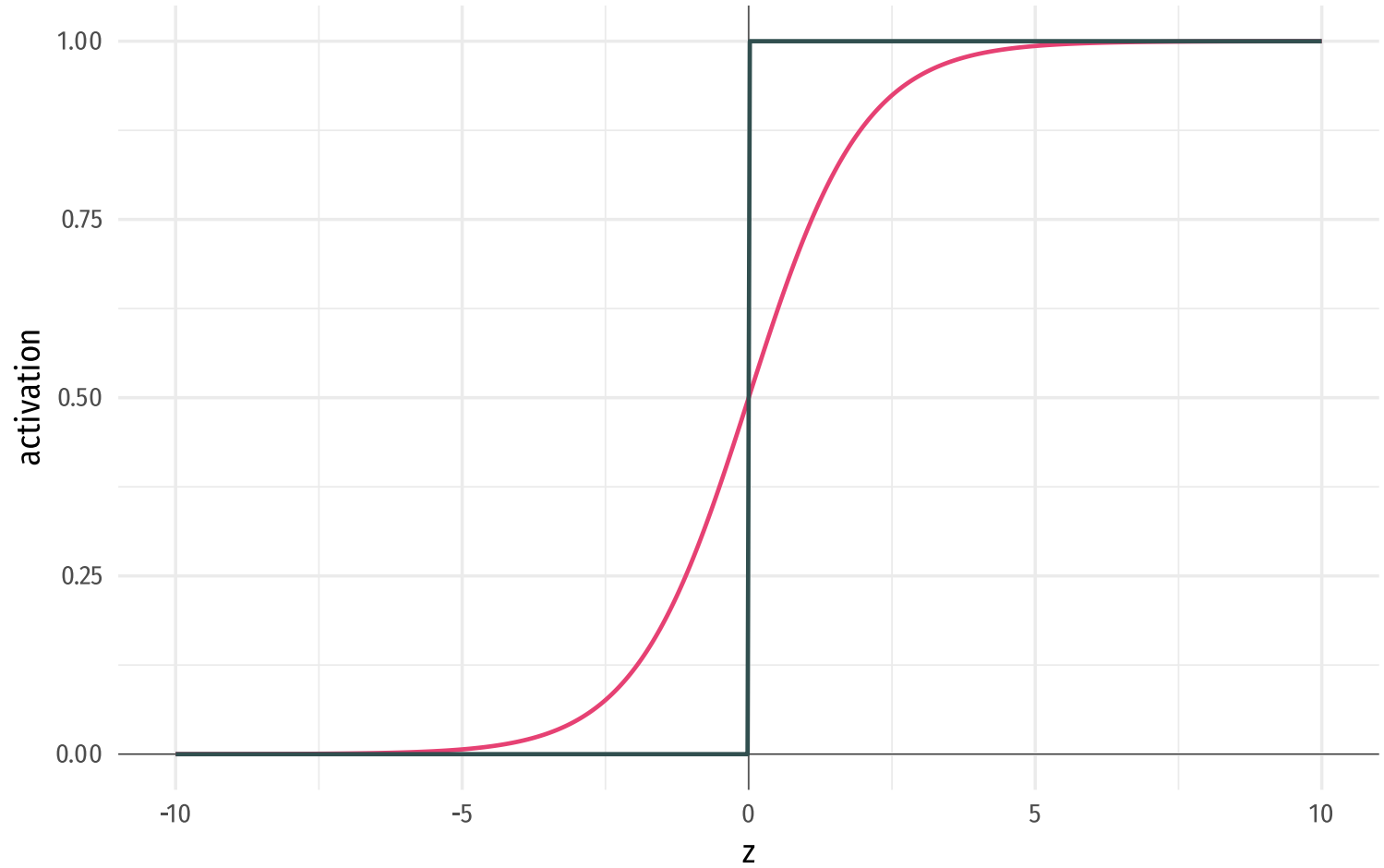
where $\sigma(z) = \frac{1}{1 + e^{-z}}$.

Also **Sigmoid neuron** with

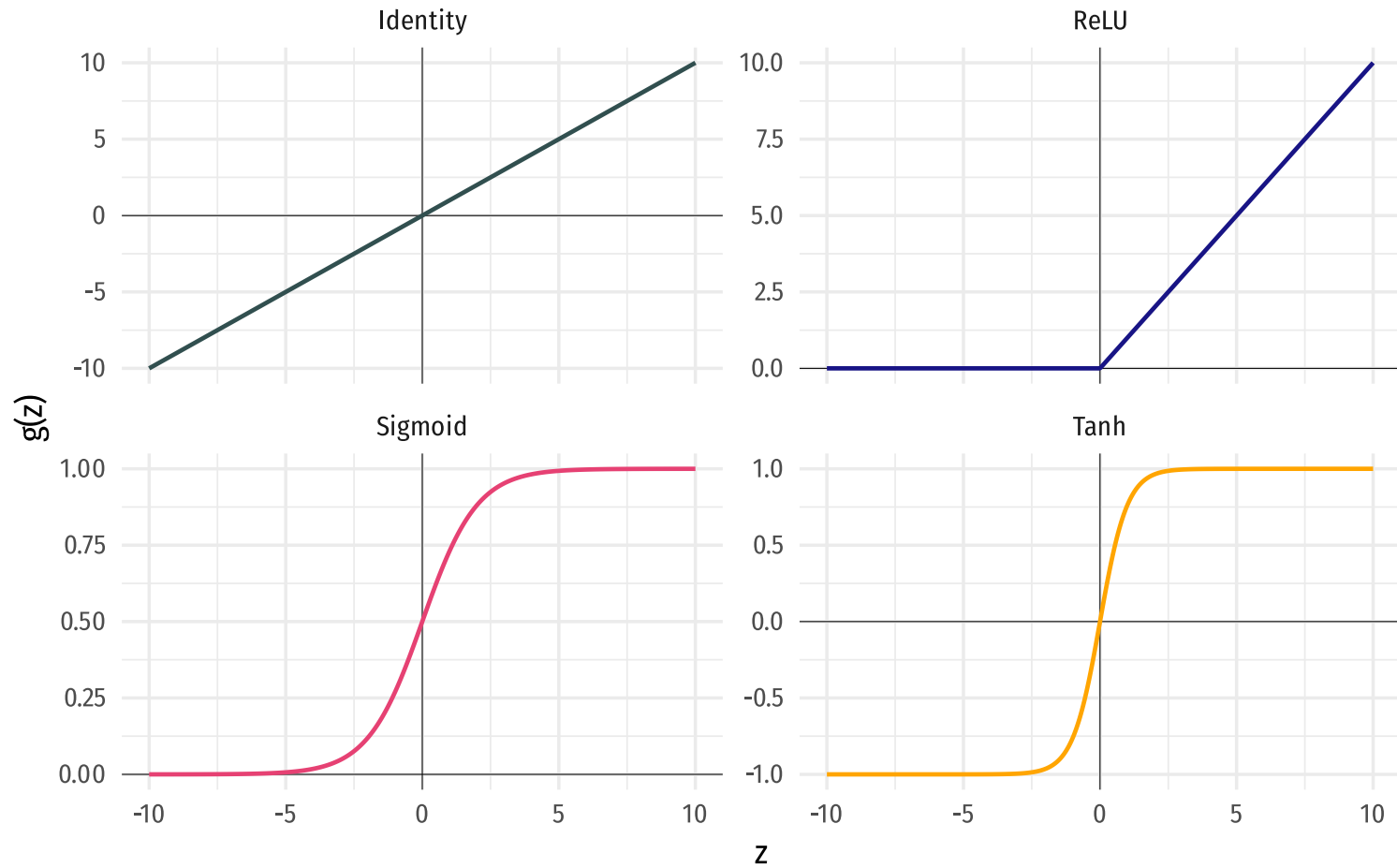
- a **weighted sum of inputs** and a **bias**,
- a **sigmoid** activation function,
- one output.

Neural networks are not a sharp break from (logistic) regression.
They are a *generalization*.

Perceptron vs. sigmoid neuron



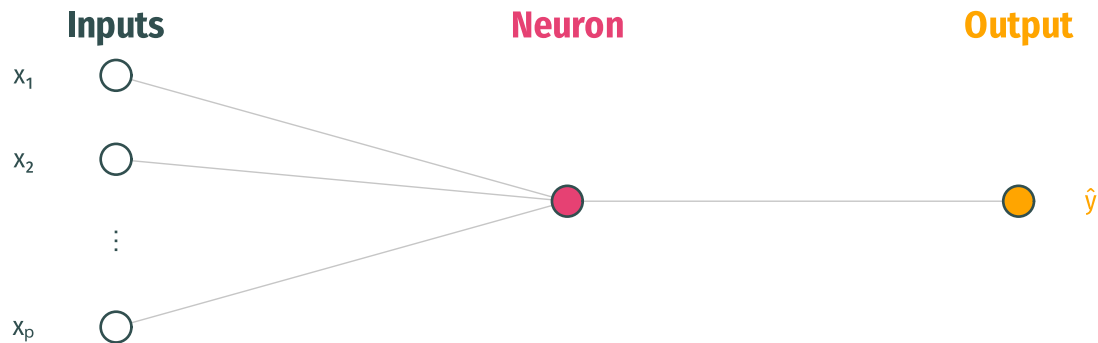
Common activation functions. (Match activation to the problem.)



Neural networks

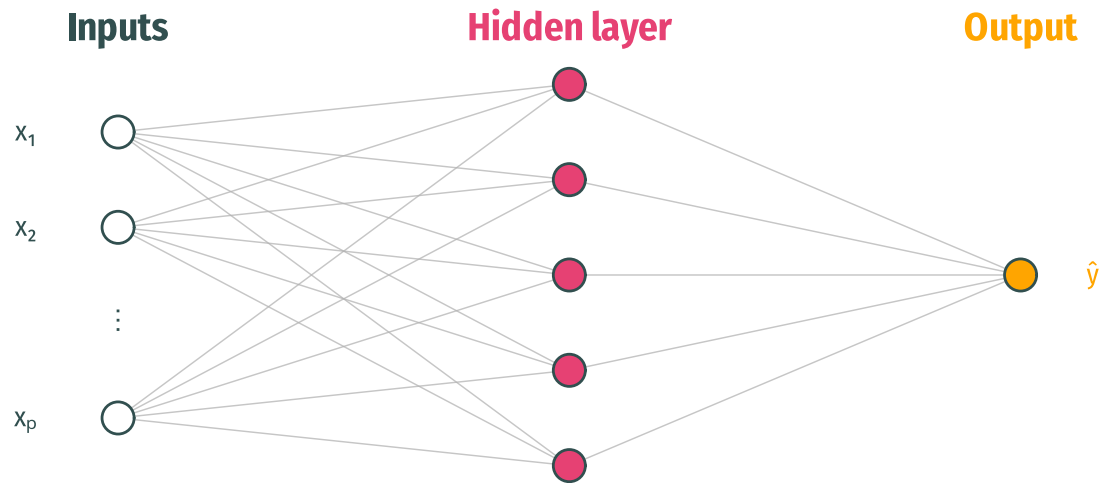
Getting flexible

Adjusting weights and bias of a **single neuron**—regardless of the activation function—is only going to get us so far. (Flavors of regression.)



Q: What else can we do to make our model *more flexible*?
(Especially for complex, highly nonlinear problems?)

Adding a **hidden layer** of neurons between the **input** and **output** layers



increases the model's ability to learn **complex relationships** from $x \rightarrow y$.

- each **hidden neuron** learns a different transformation of the input.

$$a_k = a(w'_k x + b_k)$$

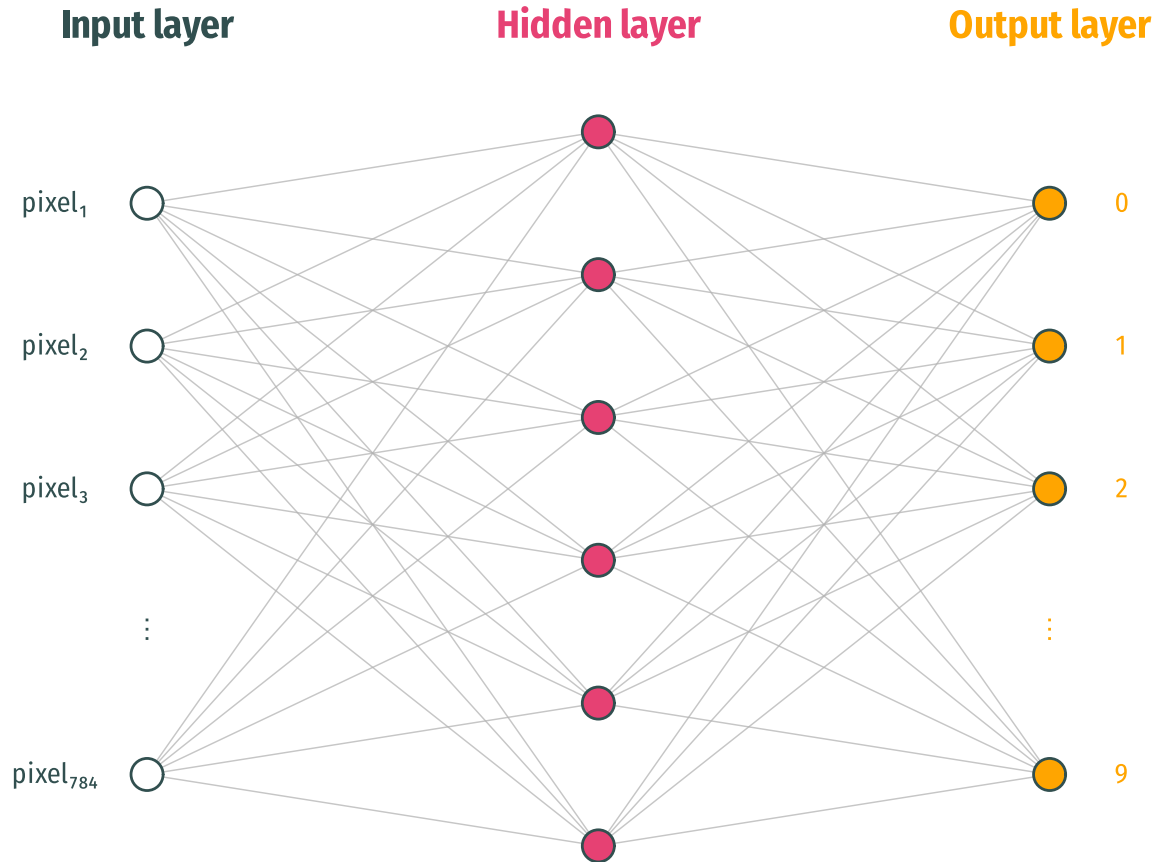
- an **output neuron** flexibly combines those transformations.

$$\hat{y} = a_y(w'_y a + b_y)$$

Let's think about building a **feed-forward network for MNIST**.

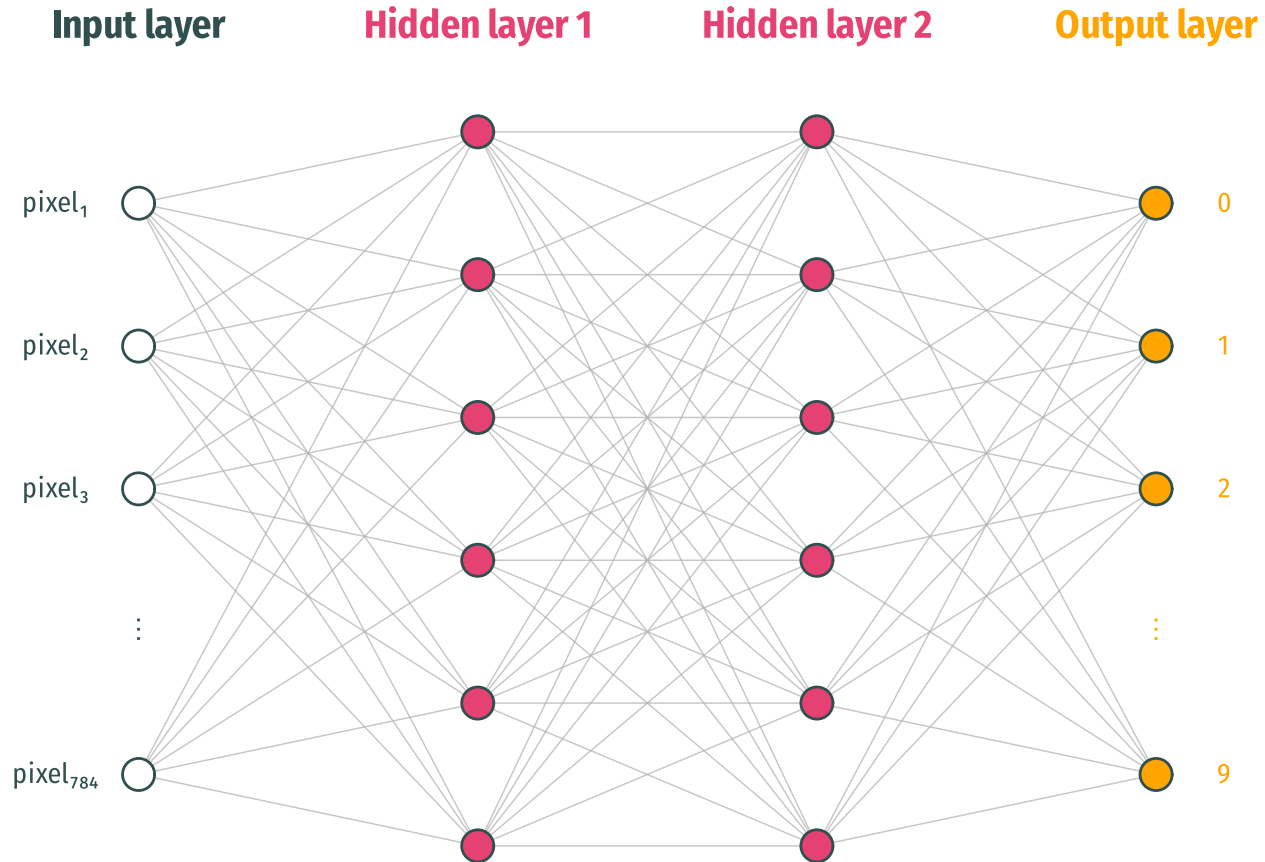
Note: **Feed-forward networks** are simply neural networks where information flows in one direction—from *input* to *output*. (No cycles or feedback loops.)

A feedforward network with one hidden layer..

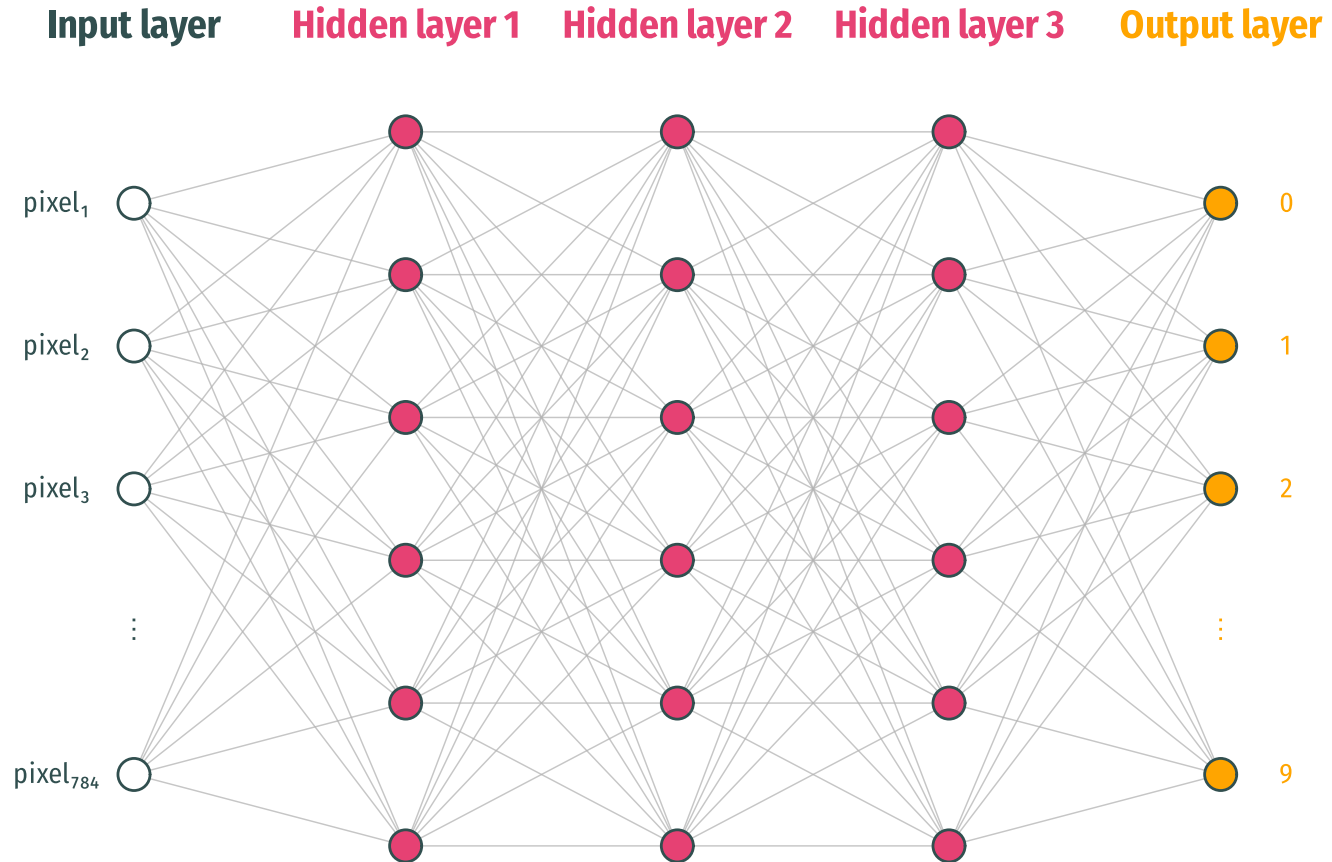


There's really no reason to stop at *one* hidden layer!

A feedforward network with two hidden layers!



A feedforward network with three hidden layers! (Getting deep!)



Neural networks

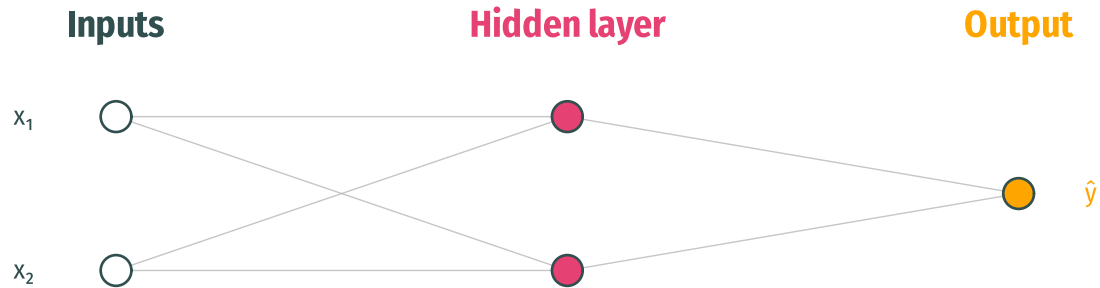
Neuron stacking in hidden layers

Let's be clear: Hidden layers are not about *more coefficients*.

The *power of hidden layers* comes from their ability to *learn increasingly complex features* built from simpler ones.

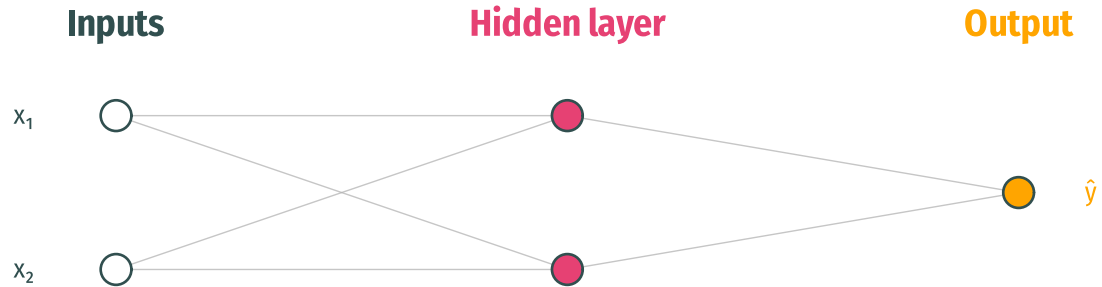
- Early neurons can respond to simple patterns;
- later neurons can combine those neurons into more complex patterns;
- every layer layers complexity on top of the previous layers.

Adding **depth** (hidden layers) allows NNs to learn more levels of *abstraction*.
(It also adds more parameters to learn.)



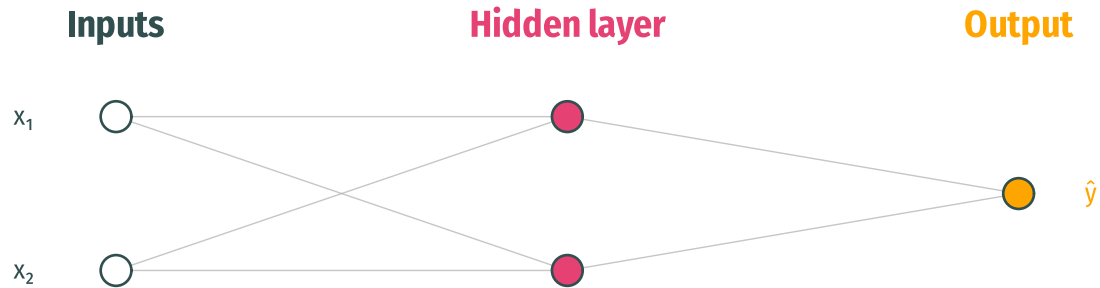
To see how *depth* and *activations* affect network flexibility and output, let's write out the math for this simple neural network.

- 2 inputs,
- 2 hidden neurons, and
- 1 output.



In the **hidden layer**, neuron a_j learns a transformation g of the *input*:

$$a_j = g(w_{j,1}^1 x_1 + w_{j,2}^1 x_2 + b_j^1)$$

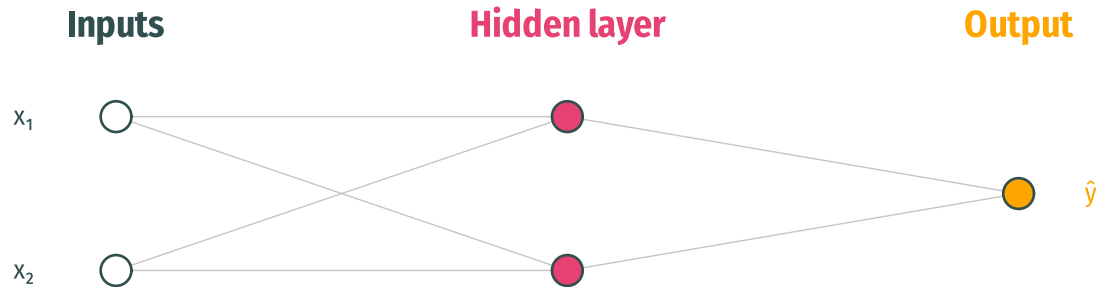


In the **hidden layer**, neuron a_j learns a transformation g of the *input*:

$$a_j = g(w_{j,1}^1 x_1 + w_{j,2}^1 x_2 + b_j^1)$$

The **output** $\hat{y}$ is a transformation h of the *hidden layer output*:

$$\hat{y} = h(w_{1,1}^2 a_1 + w_{1,2}^2 a_2 + b_1^2)$$



In the **hidden layer**, neuron a_j learns a transformation g of the *input*:

$$a_j = g(w_{j,1}^1 x_1 + w_{j,2}^1 x_2 + b_j^1)$$

The **output** $\hat{y}$ is a transformation h of the **hidden layer output**:

$$\hat{y} = h(w_{1,1}^2 a_1 + w_{1,2}^2 a_2 + b_1^2)$$

Combining the preceding two steps...

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

Case 1: Suppose g and h are both the **identity function** $f(z) = z$.

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

Case 1: Suppose g and h are both the **identity function** $f(z) = z$.

$$\hat{y} = \left(w_{1,1}^2 (w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 (w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

Case 1: Suppose g and h are both the **identity function** $f(z) = z$.

$$\begin{aligned}\hat{y} &= \left(w_{1,1}^2 (w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 (w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right) \\ &= w_{1,1}^2 w_{1,1}^1 x_1 + w_{1,1}^2 w_{1,2}^1 x_2 + w_{1,1}^2 b_1^1 + w_{1,2}^2 w_{2,1}^1 x_1 + w_{1,2}^2 w_{2,2}^1 x_2 + b_2^1 + b_1^2\end{aligned}$$

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

Case 1: Suppose g and h are both the **identity function** $f(z) = z$.

$$\begin{aligned}\hat{y} &= \left(w_{1,1}^2 (w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 (w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right) \\ &= w_{1,1}^2 w_{1,1}^1 x_1 + w_{1,1}^2 w_{1,2}^1 x_2 + w_{1,1}^2 b_1^1 + w_{1,2}^2 w_{2,1}^1 x_1 + w_{1,2}^2 w_{2,2}^1 x_2 + b_2^1 + b_1^2 \\ &= (w_{1,1}^2 w_{1,1}^1 + w_{1,2}^2 w_{2,1}^1) x_1 + (w_{1,1}^2 w_{1,2}^1 + w_{1,2}^2 w_{2,2}^1) x_2 + (w_{1,1}^2 b_1^1 + w_{1,2}^2 b_2^1 + b_1^2)\end{aligned}$$

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

Case 1: Suppose g and h are both the **identity function** $f(z) = z$.

$$\begin{aligned}\hat{y} &= \left(w_{1,1}^2 (w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 (w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right) \\ &= w_{1,1}^2 w_{1,1}^1 x_1 + w_{1,1}^2 w_{1,2}^1 x_2 + w_{1,1}^2 b_1^1 + w_{1,2}^2 w_{2,1}^1 x_1 + w_{1,2}^2 w_{2,2}^1 x_2 + b_2^1 + b_1^2 \\ &= (w_{1,1}^2 w_{1,1}^1 + w_{1,2}^2 w_{2,1}^1) x_1 + (w_{1,1}^2 w_{1,2}^1 + w_{1,2}^2 w_{2,2}^1) x_2 + (w_{1,1}^2 b_1^1 + w_{1,2}^2 b_2^1 + b_1^2) \\ &= \beta_0 + \beta_1 x_1 + \beta_2 x_2\end{aligned}$$

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

Case 1: Suppose g and h are both the **identity function** $f(z) = z$.

$$\begin{aligned}\hat{y} &= \left(w_{1,1}^2 (w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 (w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right) \\ &= w_{1,1}^2 w_{1,1}^1 x_1 + w_{1,1}^2 w_{1,2}^1 x_2 + w_{1,1}^2 b_1^1 + w_{1,2}^2 w_{2,1}^1 x_1 + w_{1,2}^2 w_{2,2}^1 x_2 + b_2^1 + b_1^2 \\ &= (w_{1,1}^2 w_{1,1}^1 + w_{1,2}^2 w_{2,1}^1) x_1 + (w_{1,1}^2 w_{1,2}^1 + w_{1,2}^2 w_{2,2}^1) x_2 + (w_{1,1}^2 b_1^1 + w_{1,2}^2 b_2^1 + b_1^2) \\ &= \beta_0 + \beta_1 x_1 + \beta_2 x_2\end{aligned}$$

A lot of work to get back to a **linear model**!

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

Case 1: Suppose g and h are both the **identity function** $f(z) = z$.

$$\begin{aligned}\hat{y} &= \left(w_{1,1}^2 (w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 (w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right) \\ &= w_{1,1}^2 w_{1,1}^1 x_1 + w_{1,1}^2 w_{1,2}^1 x_2 + w_{1,1}^2 b_1^1 + w_{1,2}^2 w_{2,1}^1 x_1 + w_{1,2}^2 w_{2,2}^1 x_2 + b_2^1 + b_1^2 \\ &= (w_{1,1}^2 w_{1,1}^1 + w_{1,2}^2 w_{2,1}^1) x_1 + (w_{1,1}^2 w_{1,2}^1 + w_{1,2}^2 w_{2,2}^1) x_2 + (w_{1,1}^2 b_1^1 + w_{1,2}^2 b_2^1 + b_1^2) \\ &= \beta_0 + \beta_1 x_1 + \beta_2 x_2\end{aligned}$$

A lot of work to get back to a **linear model**!

Case 2: Suppose g and h are **linear/affine** functions.

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

Case 1: Suppose g and h are both the **identity function** $f(z) = z$.

$$\begin{aligned}\hat{y} &= \left(w_{1,1}^2 (w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 (w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right) \\ &= w_{1,1}^2 w_{1,1}^1 x_1 + w_{1,1}^2 w_{1,2}^1 x_2 + w_{1,1}^2 b_1^1 + w_{1,2}^2 w_{2,1}^1 x_1 + w_{1,2}^2 w_{2,2}^1 x_2 + b_2^1 + b_1^2 \\ &= (w_{1,1}^2 w_{1,1}^1 + w_{1,2}^2 w_{2,1}^1) x_1 + (w_{1,1}^2 w_{1,2}^1 + w_{1,2}^2 w_{2,2}^1) x_2 + (w_{1,1}^2 b_1^1 + w_{1,2}^2 b_2^1 + b_1^2) \\ &= \beta_0 + \beta_1 x_1 + \beta_2 x_2\end{aligned}$$

A lot of work to get back to a **linear model**!

Case 2: Suppose g and h are **linear/affine** functions.

(insert math) ... Still a **linear model**!

Neural networks

Hidden layers need nonlinearity

So hidden layers only "work" when we add **nonlinear activations**.

$$\hat{y} = h\left(w_{1,1}^2 g(w_{1,1}^1 x_1 + w_{1,2}^1 x_2 + b_1^1) + w_{1,2}^2 g(w_{2,1}^1 x_1 + w_{2,2}^1 x_2 + b_2^1) + b_1^2\right)$$

If g and h are nonlinear, then $\hat{y}$ is

- a **nonlinear combination** of the **hidden-layer activations**, which are
- nonlinear combinations of the inputs.

As you add more hidden layers, you *layer* more nonlinear transformations.

Note The **output** activation function does not have to be nonlinear for the hidden layers to "work". (No requirement that g and h match either.)

Universal approximation theorem

A feedforward network with one hidden layer can approximate *any* continuous function on a compact domain, given enough neurons in the hidden layer.

(It doesn't say how many neurons we need.)

Neural networks

Welcome to the playground

TensorFlow's *Playground* is a fun way to see NNs in action.

Play with the

- activation function,
- number of hidden layers and neurons,
- learning rate,
- amount of regularization.

Watch the **boundary** change and the **representation** of hidden layers.

Neural networks

How the network learns

Neural networks

The loss function still runs the show

As always: We choose weights and biases to minimize a **loss function**.

- *regression*: MSE, MAE, Huber (*linear output*)
- *binary*: log loss (binary cross-entropy). (*sigmoid output*)

$$C = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})]$$

- *multiclass*: multiclass cross-entropy (*softmax output*)

$$C = - \sum_{k=1}^K y_k \log \hat{p}_k$$

Neural networks

The loss function still runs the show

As always: We choose weights and biases to minimize a **loss function**.

- *regression*: MSE, MAE, Huber (*linear output*)
- *binary*: log loss (binary cross-entropy). (*sigmoid output*)

$$C = -[y \log(\hat{p}) + (1 - y) \log(1 - \hat{p})]$$

- *multiclass*: multiclass cross-entropy (*softmax output*)

$$C = - \sum_{k=1}^K y_k \log \hat{p}_k$$

As before: We can **penalize** (**regularize**) the weights to prevent overfitting.

Neural networks

Optimization

We don't have time to cover the details of optimization; here's the *gist*:

- **gradient descent** move parameters "downhill" fastest by following the negative gradient of the loss function

$$\theta \leftarrow \theta - \eta \nabla_{\theta} C(\theta)$$

- **gradient** how each weight/bias changes the loss
- **backpropagation** efficient chain rule to compute those gradients
- **mini-batch** one small chunk of data used for one update
- **epoch** one full pass through all training observations

It's a lot. In fact, this *gist* barely scratches the surface.

The key is efficiently computing the gradients—backpropagation.

Neural networks

MNIST application

Neural networks

Why MNIST again?

MNIST is a natural neural-net example

- high-dimensional inputs (784 predictors for each image),
- 10 output classes,
- nonlinear relationships between pixels and classes,
- lots of variation in handwriting.

Let's see how well a neural network can learn from the raw pixel data.

The raw inputs are still images.



Neural networks

One image, many outputs

MNIST is a **multi-output classification** problem.

For each image, the output layer should produce:

$$(\hat{p}_0, \hat{p}_1, \dots, \hat{p}_9)$$

If we remove the hidden layer and keep only the softmax output,

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{k=0} e^{z_k}}$$

we are back at **multinomial logistic regression**.

Neural networks

Held-out test results

After tuning and fitting on the training data, here are the test accuracies

- *multinomial logit*: 87.8%
- *NN (1 hidden layer with 50 units)*:[†] 91.2%
- *convolutional NN*:^{††} 95.2%

Nonlinearity and achitectural choices matter

...even for a "simple" problem like MNIST.

(Simple is relative.)

[†] You can use the `brulee` package to fit a neural net using a `tidymodels` `recipe`. You'll also need to install `torch`.

^{††} I know, we didn't get to CNNs. They're an approach that *slides filters* across the input to detect local patterns.

Neural networks

Wrap-up

Neural networks

Main takeaways

NNs are a powerful class of models that extends many prior topics

- **logistic regression** is a one-neuron network with sigmoid activation,
- **hidden layers** allow NNs to learn complex representations of the data,
- **nonlinear activations** are essential for hidden layers,
- **training** involves min. loss via gradient descent and backprop.,
- **penalization** mirrors bias-variance intuition from regularized regression.

As always Careful tuning and construction are key to performance.